

UNIT 1: ALGORITHM ANALYSIS NOTES

1. INSERTION SORT

1.1 Algorithm Jist

- **Type:** Comparison-based, incremental sorting algorithm
- **How it works:** Builds a sorted subarray one element at a time. Takes one element from the unsorted portion and inserts it into its correct position in the sorted portion by shifting larger elements to the right.
- **Best case to use:**
 - Small input sizes ($n \leq 20$)
 - Nearly sorted arrays (adaptive property)
 - Online sorting (data arriving one by one)
 - When stability is required with simple implementation

1.2 Complexity Analysis

Algorithm with Costs:

```
INSERTION-SORT(A, n)
1   for i = 2 to n                -> c1 (n times)
2       key = A[i]                -> c2 (n-1 times)
3       // Insert A[i] into sorted subarray
4       j = i - 1                  -> c4 (n-1 times)
5       while j > 0 and A[j] > key -> c5 (Sum t_i from i=2 to n)
6           A[j+1] = A[j]          -> c6 (Sum(t_i-1) from i=2 to n)
7           j = j-1                -> c7 (Sum(t_i-1) from i=2 to n)
8       A[j+1] = key               -> c8 (n-1 times)
```

Where t_i = number of times while loop test executes for each i .

Worst Case (Reverse Sorted Array):

- $t_i = i$ for each i (element must move all the way to front)
- $T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5(\text{Sum } i \text{ from } 2 \text{ to } n) + c_6(\text{Sum}(i-1) \text{ from } 2 \text{ to } n) + c_7(\text{Sum}(i-1) \text{ from } 2 \text{ to } n) + c_8(n-1)$
- $T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5[n(n+1)/2 - 1] + c_6[n(n-1)/2] + c_7[n(n-1)/2] + c_8(n-1)$
- $T(n) = (c_5/2 + c_6/2 + c_7/2)n^2 + (c_1 + c_2 + c_4 + c_5/2 - c_6/2 - c_7/2 + c_8)n - (c_2 + c_4 + c_5 + c_8)$
- **$T(n) = \Theta(n^2)$**

Average Case:

- On average, $t_i = i/2$ (element moves halfway)
- $T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5(\text{Sum } i/2) + c_6(\text{Sum}(i/2 - 1)) + c_7(\text{Sum}(i/2 - 1)) + c_8(n-1)$
- Still dominated by n^2 term
- **$T(n) = \Theta(n^2)$**

Best Case (Already Sorted):

- $t_i = 1$ for all i (only test condition, no inner loop execution)
- $T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1)$
- $T(n) = (c_1 + c_2 + c_4 + c_5 + c_8) \cdot n - (c_2 + c_4 + c_5 + c_8)$
- **$T(n) = \Theta(n)$**

1.3 Properties**Stable: YES**

- **Reason:** When $A[j] > \text{key}$ (strictly greater), we shift. Equal elements are NOT shifted past each other. The relative order of equal elements is preserved because we only move when strictly greater.

In-place: YES

- **Reason:** Only uses $O(1)$ extra space (variables: key, i, j). All rearrangements happen within the original array. No auxiliary arrays needed.

Adaptive: YES

- **Reason:** Running time depends on input order. For nearly sorted arrays, inner while loop executes fewer times. Best case $\Theta(n)$ when array is already sorted.

1.4 Visual Example

Initial Array: $A = [5, 2, 4, 6, 1, 3]$, $n = 6$

Iteration i = 2:

- $\text{key} = A[2] = 2$
- $j = 1, A[1] = 5 > 2 \rightarrow A[2] = 5, j = 0$
- $A[1] = 2$
- Array: $[2, 5, 4, 6, 1, 3]$

Iteration i = 3:

- $\text{key} = A[3] = 4$
- $j = 2, A[2] = 5 > 4 \rightarrow A[3] = 5, j = 1$
- $j = 1, A[1] = 2$ not greater than 4 $\rightarrow$ stop
- $A[2] = 4$
- Array: $[2, 4, 5, 6, 1, 3]$

Iteration i = 4:

- $\text{key} = A[4] = 6$
- $j = 3, A[3] = 5$ not greater than 6 $\rightarrow$ stop immediately
- $A[4] = 6$
- Array: $[2, 4, 5, 6, 1, 3]$

Iteration i = 5:

- $\text{key} = A[5] = 1$
- $j = 4, A[4] = 6 > 1 \rightarrow A[5] = 6, j = 3$
- $j = 3, A[3] = 5 > 1 \rightarrow A[4] = 5, j = 2$
- $j = 2, A[2] = 4 > 1 \rightarrow A[3] = 4, j = 1$
- $j = 1, A[1] = 2 > 1 \rightarrow A[2] = 2, j = 0$
- $A[1] = 1$
- Array: $[1, 2, 4, 5, 6, 3]$

Iteration i = 6:

- $\text{key} = A[6] = 3$
- $j = 5, A[5] = 6 > 3 \rightarrow A[6] = 6, j = 4$
- $j = 4, A[4] = 5 > 3 \rightarrow A[5] = 5, j = 3$
- $j = 3, A[3] = 4 > 3 \rightarrow A[4] = 4, j = 2$
- $j = 2, A[2] = 2$ not greater than 3 $\rightarrow$ stop
- $A[3] = 3$
- **Final Array:** $[1, 2, 3, 4, 5, 6]$

1.5 Loop Invariant and Correctness Proof

Loop Invariant (for outer for loop): At the start of each iteration i (where $2 \leq i \leq n+1$), the subarray $A[1:i-1]$ consists of the elements originally in $A[1:i-1]$ but in sorted order.

Proof:

Initialization: Before the first iteration ($i = 2$), $A[1:1]$ contains a single element. A single element is trivially sorted. The invariant holds.

Maintenance: Assume invariant holds at start of iteration i ($A[1:i-1]$ is sorted). The algorithm stores $A[i]$ in key and finds the correct position for key in $A[1:i-1]$ by shifting elements greater than key one position right. It then inserts key at the correct position. Thus, $A[1:i]$ is sorted. Incrementing i preserves the invariant.

Termination: The loop terminates when $i = n+1$. By the invariant, $A[1:n]$ is sorted. Since $A[1:n]$ is the entire array, the array is sorted. QED

2. SELECTION SORT

2.1 Algorithm Jist

- **Type:** Comparison-based, selection algorithm
- **How it works:** Divides array into sorted and unsorted regions. Repeatedly finds the minimum element from the unsorted region and swaps it with the first element of the unsorted region, expanding the sorted region by one.
- **Best case to use:**
 - When memory writes are expensive (only $O(n)$ swaps)

- Small datasets where simplicity matters
- When checking all elements is necessary anyway

2.2 Complexity Analysis

Algorithm with Costs:

```

SELECTION-SORT(A, n)
1  for i = 1 to n - 1          -> c1 (n times)
2      min_index = i           -> c2 (n-1 times)
3      for j = i + 1 to n      -> c3 (Sum(n-i+1) from i=1 to n-1)
4          if A[j] < A[min_index] -> c4 (Sum(n-i) from i=1 to n-1)
5              min_index = j    -> c5 (Sum(t_i) from i=1 to n-1)
6      exchange A[i] with A[min_index] -> c6 (n-1 times)

```

Where t_i = number of times `min_index` is updated in iteration i .

Worst Case:

- Line 3 executes: $\text{Sum}(n-i+1)$ from $i=1$ to $n-1 = n + (n-1) + \dots + 2 = n(n+1)/2 - 1$
- Line 4 executes: $\text{Sum}(n-i)$ from $i=1$ to $n-1 = (n-1) + (n-2) + \dots + 1 = n(n-1)/2$
- Line 5 executes maximum $n-1$ times (decreasing sequence)
- $T(n) = c1n + c2(n-1) + c3[n(n+1)/2 - 1] + c4[n(n-1)/2] + c5(n-1) + c6(n-1)$
- $T(n) = [(c3 + c4)/2]n^2 + [c1 + c2 + c3/2 - c4/2 + c5 + c6/n - [c2 + c3 + c5 + c6]]n$
- $T(n) = \Theta(n^2)$

Average Case:

- Same number of comparisons regardless of input order
- Line 3 and 4 always execute same number of times
- Line 5 executes on average less than $n-1$ times
- Still dominated by n^2 term from comparisons
- $T(n) = \Theta(n^2)$

Best Case:

- Even if array is sorted, we still perform all comparisons
- Same number of operations as worst case
- $T(n) = \Theta(n^2)$

2.3 Properties

Stable: NO

- **Reason:** The swap operation can change the relative order of equal elements. Example: $[4a, 4b, 3]$. `min_index` finds 3 at position 3, swaps with position 1: $[3, 4b, 4a]$. The two 4s have reversed their relative order.

In-place: YES

- **Reason:** Uses only $O(1)$ extra space (variables: `i`, `j`, `min_index`). All operations are swaps within the original array. No auxiliary storage needed.

Adaptive: NO

- **Reason:** Running time is $\Theta(n^2)$ regardless of input order. Even if array is already sorted, the algorithm performs all $n(n-1)/2$ comparisons. No performance improvement for partially sorted input.

2.4 Visual Example

Initial Array: $A = [64, 25, 12, 22, 11]$, $n = 5$

Iteration i = 1:

- $\text{min_index} = 1$ ($A[1] = 64$)
- $j = 2$: $A[2] = 25 < 64 \rightarrow \text{min_index} = 2$
- $j = 3$: $A[3] = 12 < 25 \rightarrow \text{min_index} = 3$
- $j = 4$: $A[4] = 22$ not less than 12 $\rightarrow$ no change
- $j = 5$: $A[5] = 11 < 12 \rightarrow \text{min_index} = 5$
- Swap $A[1]$ (64) with $A[5]$ (11)
- Array: $[11, 25, 12, 22, 64]$

Iteration i = 2:

- $\text{min_index} = 2$ ($A[2] = 25$)
- $j = 3$: $A[3] = 12 < 25 \rightarrow \text{min_index} = 3$
- $j = 4$: $A[4] = 22$ not less than 12 $\rightarrow$ no change
- $j = 5$: $A[5] = 64$ not less than 12 $\rightarrow$ no change
- Swap $A[2]$ (25) with $A[3]$ (12)
- Array: $[11, 12, 25, 22, 64]$

Iteration i = 3:

- $\text{min_index} = 3$ ($A[3] = 25$)
- $j = 4$: $A[4] = 22 < 25 \rightarrow \text{min_index} = 4$
- $j = 5$: $A[5] = 64$ not less than 22 $\rightarrow$ no change
- Swap $A[3]$ (25) with $A[4]$ (22)
- Array: $[11, 12, 22, 25, 64]$

Iteration i = 4:

- $\text{min_index} = 4$ ($A[4] = 25$)
- $j = 5$: $A[5] = 64$ not less than 25 $\rightarrow$ no change
- Swap $A[4]$ with $A[4]$ (no change)
- **Final Array:** $[11, 12, 22, 25, 64]$

2.5 Loop Invariant and Correctness Proof

Loop Invariant (for outer for loop): At the start of each iteration i (where $1 \leq i \leq n$), the subarray $A[1:i-1]$ contains the $i-1$ smallest elements of the original array in sorted order.

Proof:

Initialization: Before first iteration ($i = 1$), $A[1:0]$ is empty. Empty subarray trivially contains 0 smallest elements in sorted order. Invariant holds.

Maintenance: Assume invariant holds at start of iteration i ($A[1:i-1]$ is sorted). The inner loop finds the minimum in $A[i:n]$. This minimum is the i -th smallest overall element. Swapping it to position i ensures $A[1:i]$ contains i smallest elements sorted. Incrementing i preserves invariant.

Termination: Loop terminates when $i = n$. By invariant, $A[1:n-1]$ contains $n-1$ smallest elements sorted. The remaining element at $A[n]$ must be the largest. Thus $A[1:n]$ is sorted. QED

3. MERGE SORT

3.1 Algorithm Jist

- **Type:** Comparison-based, divide-and-conquer sorting algorithm
- **How it works:** Recursively divides the array into halves until each subarray has one element, then merges subarrays back together in sorted order.
- **Best case to use:**
 - Large datasets where $O(n \log n)$ guarantee is needed
 - Linked lists (efficient merging without extra space for pointers)
 - External sorting (data too large for memory)
 - When stability is required with optimal worst-case performance

3.2 Complexity Analysis

MERGE with Costs:

```
MERGE(A, p, q, r)
1  n_L = q - p + 1          -> c1
2  n_R = r - q              -> c2
3  let L[0:n_L-1] and R[0:n_R-1] be new -> c3
4  for i = 0 to n_L - 1      -> c4 (n_L + 1 times)
5      L[i] = A[p + i]       -> c5 (n_L times)
6  for j = 0 to n_R - 1      -> c6 (n_R + 1 times)
7      R[j] = A[q + j + 1]   -> c7 (n_R times)
8  i = 0                     -> c8
9  j = 0                     -> c9
10 k = p                     -> c10
12 while i < n_L and j < n_R  -> c11 (at most n_L + n_R + 1 times)
13     if L[i] <= R[j]        -> c12 (at most n_L + n_R times)
14         A[k] = L[i]        -> c13
15         i = i + 1          -> c14
16     else A[k] = R[j]       -> c15
17         j = j + 1          -> c16
18     k = k + 1              -> c17
```

```

20  while i < n_L                                -> c18 (at most n_L + 1 times)
21      A[k] = L[i]                              -> c19
22      i = i + 1                                -> c20
23      k = k + 1                                -> c21
24  while j < n_R                                -> c22 (at most n_R + 1 times)
25      A[k] = R[j]                              -> c23
26      j = j + 1                                -> c24
27      k = k + 1                                -> c25

```

MERGE Time: $\Theta(n)$ where $n = r - p + 1$ (total elements merged)

MERGE-SORT with Recurrence:

```

MERGE-SORT(A, p, r)
1  if p <= r                                    -> c1
2      return                                    -> c2
3  q = floor((p + r)/2)                        -> c3
4  MERGE-SORT(A, p, q)                         -> T(n/2)
5  MERGE-SORT(A, q + 1, r)                     -> T(n/2)
7  MERGE(A, p, q, r)                           -> Theta(n)

```

Recurrence Relation: $T(n) = \Theta(1)$ if $n = 1$ $T(n) = 2T(n/2) + \Theta(n)$ if $n > 1$

Solving by Recursion Tree:

- Level 0: cn (work at root)
- Level 1: $2 * c(n/2) = cn$
- Level 2: $4 * c(n/4) = cn$
- ...
- Level k : $2^k * c(n/2^k) = cn$

Number of levels = $\log_2(n)$ (since $n/2^k = 1 \rightarrow k = \log_2(n)$)

Total work = $cn * \log_2(n) = \Theta(n \log n)$

By Master Theorem: $a = 2$, $b = 2$, $f(n) = \Theta(n)$ $n^{\log_b a} = n^{\log_2 2} = n^1 = n$ Since $f(n) = \Theta(n) = \Theta(n^{\log_b a})$, Case 2 applies. **$T(n) = \Theta(n \log n)$** for all cases (worst, average, best).

3.3 Properties

Stable: YES

- **Reason:** In MERGE, when $L[i] \leq R[j]$, we take from L first. Equal elements from the left subarray (which appeared first in the original array) are placed before elements from the right subarray. The $\leq$ ensures stability.

In-place: NO

- **Reason:** MERGE requires auxiliary arrays L and R of total size $n_L + n_R = n$. Extra space requirement is $\Theta(n)$. Not in-place.

Adaptive: NO

- **Reason:** Always divides into halves and merges regardless of input order. Running time is $\Theta(n \log n)$ even for sorted arrays. No performance improvement for partially sorted input.

3.4 Visual Example

Initial Array: $A = [38, 27, 43, 3, 9, 82, 10]$, $n = 7$

Divide Phase:

```
[38, 27, 43, 3, 9, 82, 10]
      /           \
[38, 27, 43]      [3, 9, 82, 10]
    /  \         /  \
[38]   [27, 43] [3, 9] [82, 10]
          /  \   /  \   /  \
        [27] [43] [3] [9]  [82] [10]
```

Conquer/Merge Phase (bottom-up):

Merge [27] and [43]:

- $L = [27]$, $R = [43]$
- $L[0] = 27 \leq 43 = R[0] \rightarrow A[0] = 27$
- L exhausted $\rightarrow A[1] = 43$
- Result: $[27, 43]$

Merge [38] and [27, 43]:

- $L = [38]$, $R = [27, 43]$
- $L[0] = 38 > 27 = R[0] \rightarrow A[0] = 27$
- $L[0] = 38 > 43$? No, $38 \leq 43 \rightarrow A[1] = 38$
- L exhausted $\rightarrow A[2] = 43$
- Result: $[27, 38, 43]$

Merge [3] and [9]:

- Result: $[3, 9]$

Merge [82] and [10]:

- $L = [82]$, $R = [10]$
- $L[0] = 82 > 10 = R[0] \rightarrow A[0] = 10$
- R exhausted $\rightarrow A[1] = 82$
- Result: $[10, 82]$

Merge [3, 9] and [10, 82]:

- $L = [3, 9]$, $R = [10, 82]$
- $L[0] = 3 \leq 10 \rightarrow A[0] = 3$

- $L[0] = 9 \leq 10 \rightarrow A[1] = 9$
- L exhausted $\rightarrow A[2] = 10, A[3] = 82$
- Result: $[3, 9, 10, 82]$

Final Merge [27, 38, 43] and [3, 9, 10, 82]:

- $L = [27, 38, 43], R = [3, 9, 10, 82]$
- $L[0] = 27 > 3 \rightarrow A[0] = 3$
- $L[0] = 27 > 9 \rightarrow A[1] = 9$
- $L[0] = 27 > 10 \rightarrow A[2] = 10$
- $L[0] = 27 \leq 82 \rightarrow A[3] = 27$
- $L[0] = 38 \leq 82 \rightarrow A[4] = 38$
- $L[0] = 43 \leq 82 \rightarrow A[5] = 43$
- L exhausted $\rightarrow A[6] = 82$
- **Final Array: [3, 9, 10, 27, 38, 43, 82]**

3.5 Loop Invariant and Correctness Proof

Loop Invariant for MERGE (while $i < n_L$ and $j < n_R$): At the start of each iteration, $A[p:k-1]$ contains the $k-p$ smallest elements of $L[0:n_L-1]$ and $R[0:n_R-1]$ in sorted order. $L[i]$ and $R[j]$ are the smallest elements of their arrays not yet copied to A .

Proof of MERGE correctness:

Initialization: Before first iteration, $k = p$, so $A[p:p-1]$ is empty. Empty subarray trivially contains 0 smallest elements. $L[0]$ and $R[0]$ are smallest in their arrays. Invariant holds.

Maintenance: Assume invariant holds. If $L[i] \leq R[j]$, then $L[i]$ is the smallest remaining element overall. Copying it to $A[k]$ maintains sorted order. Incrementing i and k preserves invariant. Similarly if $R[j] < L[i]$.

Termination: When loop terminates, one array is exhausted. The remaining elements in the other array are all larger than those in $A[p:k-1]$ and are already sorted. The final while loops copy them directly, maintaining sorted order.

MERGE-SORT correctness by induction: Base case: $n = 1$, array is trivially sorted. Inductive step: Assume MERGE-SORT works for all sizes $< n$. For size n , it sorts two halves of size $n/2$ correctly (by IH), then MERGE correctly combines them. Thus MERGE-SORT is correct. QED

4. BUBBLE SORT

4.1 Algorithm Jist

- **Type:** Comparison-based, exchange sorting algorithm
- **How it works:** Repeatedly steps through the array, compares adjacent elements, and swaps them if they are in the wrong order. Each pass “bubbles” the largest unsorted element to its correct position at the end.

- **Best case to use:**
 - Educational purposes (simple to understand)
 - Nearly sorted arrays with optimization (adaptive variant)
 - Small datasets where code simplicity matters

4.2 Complexity Analysis

Algorithm with Costs:

```
BUBBLESORT(A, n)
  for i = 1 to n - 1                -> c1 (n times)
    for j = n downto i + 1          -> c2 (Sum(n-i) from i=1 to n-1 + (n-1))
      if A[j] < A[j - 1]            -> c3 (Sum(n-i) from i=1 to n-1)
        exchange A[j] with A[j-1]  -> c4 (Sum(t_i) from i=1 to n-1)
```

Where t_i = number of swaps in iteration i .

Worst Case (Reverse Sorted):

- Every comparison results in a swap
- Line 2 executes: $\text{Sum}(n-i+1)$ from $i=1$ to $n-1 = n + (n-1) + \dots + 2 = n(n+1)/2 - 1$
- Line 3 executes: $\text{Sum}(n-i)$ from $i=1$ to $n-1 = (n-1) + \dots + 1 = n(n-1)/2$
- Line 4 executes: $n(n-1)/2$ times
- $T(n) = c_1n + c_2[n(n+1)/2 - 1] + c_3[n(n-1)/2] + c_4[n(n-1)/2]$
- **$T(n) = \Theta(n^2)$**

Average Case:

- About half the comparisons result in swaps
- Still $\Theta(n^2)$ comparisons
- **$T(n) = \Theta(n^2)$**

Best Case (Already Sorted):

- No swaps needed, but all comparisons still performed
- $T(n) = c_1n + c_2[n(n+1)/2 - 1] + c_3[n(n-1)/2]$
- **$T(n) = \Theta(n^2)$** (without optimization)

Note: With a flag to detect if any swap occurred, best case can be optimized to $\Theta(n)$.

4.3 Properties

Stable: YES

- **Reason:** We only swap when $A[j] < A[j-1]$ (strictly less). When equal elements are adjacent, no swap occurs. The relative order of equal elements is preserved.

In-place: YES

- **Reason:** Only uses $O(1)$ extra space (variables: i, j , temp for swap). All operations are swaps within the original array.

Adaptive: NO (basic version), YES (optimized version)

- **Reason:** Basic version performs all $n(n-1)/2$ comparisons regardless of input. However, with an optimization flag that stops early if no swaps occur in a pass, it becomes adaptive with best case $\Theta(n)$ for sorted arrays.

4.4 Visual Example

Initial Array: $A = [5, 1, 4, 2, 8]$, $n = 5$

Pass i = 1: (j goes from 5 down to 2)

- $j = 5$: $A[5] = 8$ not less than $A[4] = 2$ -> no swap
- $j = 4$: $A[4] = 2 < A[3] = 4$ -> swap -> $[5, 1, 2, 4, 8]$
- $j = 3$: $A[3] = 2 < A[2] = 1$? No -> no swap
- $j = 2$: $A[2] = 1 < A[1] = 5$ -> swap -> $[1, 5, 2, 4, 8]$
- After Pass 1: $[1, 5, 2, 4, 8]$ (8 is in place)

Pass i = 2: (j goes from 5 down to 3)

- $j = 5$: $A[5] = 8$ not less than $A[4] = 4$ -> no swap
- $j = 4$: $A[4] = 4 < A[3] = 2$? No -> no swap
- $j = 3$: $A[3] = 2 < A[2] = 5$ -> swap -> $[1, 2, 5, 4, 8]$
- After Pass 2: $[1, 2, 5, 4, 8]$ (8, 5 in place)

Pass i = 3: (j goes from 5 down to 4)

- $j = 5$: $A[5] = 8$ not less than $A[4] = 4$ -> no swap
- $j = 4$: $A[4] = 4 < A[3] = 5$ -> swap -> $[1, 2, 4, 5, 8]$
- After Pass 3: $[1, 2, 4, 5, 8]$ (8, 5, 4 in place)

Pass i = 4: (j goes from 5 down to 5)

- $j = 5$: $A[5] = 8$ not less than $A[4] = 5$ -> no swap
- **Final Array:** $[1, 2, 4, 5, 8]$

4.5 Loop Invariant and Correctness Proof

Loop Invariant (outer for loop): At the start of each iteration i (where $1 \leq i \leq n$), the subarray $A[n-i+2:n]$ contains the $i-1$ largest elements of the array in sorted order.

Proof:

Initialization: Before first iteration ($i = 1$), $A[n+1:n]$ is empty. Empty subarray trivially satisfies the invariant.

Maintenance: During iteration i , the inner loop bubbles the largest element among $A[1:n-i+1]$ to position $n-i+1$. This element is at least as large as all elements in $A[n-i+2:n]$. Thus $A[n-i+1:n]$ contains i largest elements sorted.

Termination: When $i = n$, $A[2:n]$ contains $n-1$ largest elements sorted. $A[1]$ must be the smallest element. Thus entire array is sorted. QED

5. HEAPSORT

5.1 Algorithm Jist

- **Type:** Comparison-based, selection sorting using heap data structure
- **How it works:** First builds a max-heap from the array. Repeatedly extracts the maximum element (root) and places it at the end of the array, then restores the heap property on the remaining elements.
- **Best case to use:**
 - When guaranteed $O(n \log n)$ worst-case is needed without extra space
 - Priority queue implementations
 - Systems with limited memory (in-place)
 - When finding k largest/smallest elements

5.2 Complexity Analysis

Algorithm with Costs:

```
HEAPSORT(A, n)
1  BUILD-MAX-HEAP(A, n)           -> Theta(n)
2  for i = n downto 2             -> c1 (n-1 times)
3      exchange A[1] with A[i]     -> c2 (n-1 times)
4      A.heap-size = A.heap-size - 1 -> c3 (n-1 times)
5      MAX-HEAPIFY(A, i)           -> O(log n) each, (n-1) times
```

BUILD-MAX-HEAP:

- Calls MAX-HEAPIFY on $n/2$ nodes
- Time: $O(n)$ (tight bound, not $O(n \log n)$)

MAX-HEAPIFY(A, i):

- Compares $A[i]$ with left and right children
- If violation, swaps with larger child and recurses
- Time: $O(h)$ where h is height of subtree = $O(\log n)$

Recurrence for MAX-HEAPIFY: $T(n) \leq T(2n/3) + \Theta(1)$ By Master Theorem: $a=1$, $b=3/2$, $f(n)=\Theta(1)$ $n^{\log_{3/2} 1} = n^0 = 1$ Case 2: $T(n) = \Theta(\log n)$

Total HEAPSORT Time:

- BUILD-MAX-HEAP: $\Theta(n)$
- $n-1$ iterations of MAX-HEAPIFY: $(n-1) * O(\log n) = O(n \log n)$
- **$T(n) = \Theta(n \log n)$** for all cases

5.3 Properties

Stable: NO

- **Reason:** The swap of $A[1]$ with $A[i]$ can change relative order of equal elements. When the root (max element) is swapped to the end, it may jump over elements equal to it, disrupting stability.

In-place: YES

- **Reason:** Uses only $O(1)$ extra space (variables for swapping and heap operations). The heap is built within the original array. The sorted portion grows from the end while the heap occupies the prefix.

Adaptive: NO

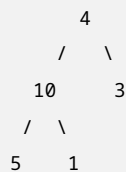
- **Reason:** Always builds a max-heap and performs $n-1$ extractions. Running time is $\Theta(n \log n)$ regardless of input order. Even for sorted arrays, the full heap construction and extraction process is executed.

5.4 Visual Example

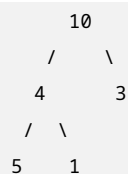
Initial Array: $A = [4, 10, 3, 5, 1]$, $n = 5$

BUILD-MAX-HEAP Phase:

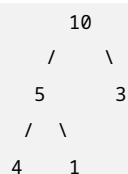
Array as complete binary tree:



- Start from $n/2 = 2$ (node with value 10):
 - 10 has children 5 and 1. 10 is largest. No change.
- Move to node 1 (value 4):
 - 4 has children 10 and 3. 10 is largest.
 - Swap 4 and 10:



- Now 4 has children 5 and 1. 5 is largest.
- Swap 4 and 5:



Max-heap built: $[10, 5, 3, 4, 1]$

HEAPSORT Extraction Phase:

i = 5:

- Swap A[1] (10) with A[5] (1)
- Array: [1, 5, 3, 4, 10], heap-size = 4
- MAX-HEAPIFY(A, 1):
- 1 has children 5 and 3. 5 is largest.
- Swap 1 and 5:

```

      5
     / \
    1   3
   /
  4

```

- 1 has child 4. $4 > 1$. Swap:

```

      5
     / \
    4   3
   /
  1

```

- Array: [5, 4, 3, 1, 10]

i = 4:

- Swap A[1] (5) with A[4] (1)
- Array: [1, 4, 3, 5, 10], heap-size = 3
- MAX-HEAPIFY(A, 1):
- 1 has children 4 and 3. 4 is largest.
- Swap 1 and 4:

```

      4
     / \
    1   3

```

- Array: [4, 1, 3, 5, 10]

i = 3:

- Swap A[1] (4) with A[3] (3)
- Array: [3, 1, 4, 5, 10], heap-size = 2
- MAX-HEAPIFY(A, 1):
- 3 has child 1. $3 > 1$. No swap needed.
- Array: [3, 1, 4, 5, 10]

i = 2:

- Swap A[1] (3) with A[2] (1)
- Array: [1, 3, 4, 5, 10], heap-size = 1

- MAX-HEAPIFY(A, 1): heap-size = 1, no children. Done.

Final Array: [1, 3, 4, 5, 10]

5.5 Loop Invariant and Correctness Proof

Loop Invariant (for HEAPSORT for loop): At the start of each iteration i (where $2 \leq i \leq n+1$), the subarray $A[1:i-1]$ is a max-heap containing the $i-1$ smallest elements, and $A[i:n]$ contains the $n-i+1$ largest elements of the original array in sorted order.

Proof:

Initialization: Before first iteration ($i = n$), BUILD-MAX-HEAP ensures $A[1:n]$ is a max-heap. $A[n+1:n]$ is empty. Invariant holds.

Maintenance: At start of iteration i , $A[1:i]$ is a max-heap. $A[1]$ is the maximum of $A[1:i]$. Swapping it to $A[i]$ places the $(n-i+1)$ -th largest element in its final position. Reducing heap-size and calling MAX-HEAPIFY restores the heap property for $A[1:i-1]$.

Termination: When $i = 1$, $A[2:n]$ contains $n-1$ largest elements sorted. $A[1]$ is the smallest element. Thus $A[1:n]$ is sorted. QED

6. QUICKSORT

6.1 Algorithm Jist

- **Type:** Comparison-based, divide-and-conquer sorting algorithm
- **How it works:** Selects a 'pivot' element and partitions the array such that elements smaller than pivot are on the left and larger elements on the right. Recursively sorts the subarrays.
- **Best case to use:**
 - General-purpose sorting (fastest in practice for most inputs)
 - When average-case performance matters more than worst-case
 - Cache-friendly applications (good locality of reference)
 - When in-place sorting with $O(\log n)$ stack space is acceptable

6.2 Complexity Analysis

PARTITION with Costs:

```

PARTITION(A, p, r)
1  x = A[r]                                -> c1
2  i = p - 1                                -> c2
3  for j = p to r - 1                        -> c3 (n times, where n = r-p+1)
4      if A[j] <= x                            -> c4 (n-1 times)
5          i = i + 1                            -> c5 (t times)
6          exchange A[i] with A[j]              -> c6 (t times)
7  exchange A[i+1] with A[r]                  -> c7
8  return i + 1                               -> c8

```

PARTITION Time: $\Theta(n)$ where $n = r - p + 1$

QUICKSORT Recurrence:

```
QUICKSORT(A, p, r)
1   if p < r                                -> c1
3       q = PARTITION(A, p, r)                -> Theta(n)
4       QUICKSORT(A, p, q-1)                  -> T(k)
5       QUICKSORT(A, q+1, r)                  -> T(n-k-1)
```

Where $k = q - p$ (number of elements in left partition).

Worst Case (Unbalanced Partitions):

- One partition has $n-1$ elements, other has 0
- $T(n) = T(n-1) + T(0) + \Theta(n)$
- $T(n) = T(n-1) + \Theta(n)$
- Unfolding: $T(n) = \Theta(n) + \Theta(n-1) + \dots + \Theta(1) = \Theta(n^2)$
- **$T(n) = \Theta(n^2)$**

Best Case (Balanced Partitions):

- Partitions are roughly equal: $k \sim n/2$
- $T(n) = 2T(n/2) + \Theta(n)$
- Same as Merge Sort: **$T(n) = \Theta(n \log n)$**

Average Case:

- Assume all splits are equally likely
- $T(n) = (2/n) \sum T(k) \text{ from } k=0 \text{ to } n-1 + \Theta(n)$
- Solving: **$T(n) = \Theta(n \log n)$**

6.3 Properties

Stable: NO

- **Reason:** The exchange operations in PARTITION can change relative order of equal elements. When $A[j] \leq x$, we swap elements that may jump over equal elements, disrupting stability.

In-place: YES

- **Reason:** PARTITION rearranges elements within the array using only $O(1)$ extra space. Recursion uses $O(\log n)$ stack space in best/average case, $O(n)$ in worst case.

Adaptive: NO

- **Reason:** Running time depends on pivot selection and partition balance, not on input order. Even sorted arrays can cause worst-case behavior with poor pivot choice (e.g., always choosing last element).

6.4 Visual Example

Initial Array: $A = [10, 80, 30, 90, 40, 50, 70]$, $p = 1$, $r = 7$

First PARTITION call:

- $x = A[7] = 70$ (pivot)
- $i = 0$

j = 1: $A[1] = 10 \leq 70 \rightarrow i = 1$, swap $A[1]$ with $A[1] \rightarrow [10, 80, 30, 90, 40, 50, 70]$

j = 2: $A[2] = 80$ not $\leq 70 \rightarrow$ no change

j = 3: $A[3] = 30 \leq 70 \rightarrow i = 2$, swap $A[2]$ (80) with $A[3]$ (30) $\rightarrow [10, 30, 80, 90, 40, 50, 70]$

j = 4: $A[4] = 90$ not $\leq 70 \rightarrow$ no change

j = 5: $A[5] = 40 \leq 70 \rightarrow i = 3$, swap $A[3]$ (80) with $A[5]$ (40) $\rightarrow [10, 30, 40, 90, 80, 50, 70]$

j = 6: $A[6] = 50 \leq 70 \rightarrow i = 4$, swap $A[4]$ (90) with $A[6]$ (50) $\rightarrow [10, 30, 40, 50, 80, 90, 70]$

End of loop: Swap $A[5]$ (80) with $A[7]$ (70) $\rightarrow [10, 30, 40, 50, 70, 90, 80]$

- Return $q = 5$

Recursive calls:

- **QUICKSORT**(A, 1, 4): $[10, 30, 40, 50] \rightarrow$ already sorted
- **QUICKSORT**(A, 6, 7): $[90, 80]$
- **PARTITION**: $x = 80, i = 5$
- **j = 6:** $A[6] = 90$ not $\leq 80 \rightarrow$ no change
- Swap $A[6]$ (90) with $A[7]$ (80) $\rightarrow [10, 30, 40, 50, 70, 80, 90]$
- Return $q = 6$
- **QUICKSORT**(A, 6, 5): empty
- **QUICKSORT**(A, 7, 7): single element

Final Array: $[10, 30, 40, 50, 70, 80, 90]$

6.5 Loop Invariant and Correctness Proof

Loop Invariant for PARTITION (for loop): At the start of each iteration j (where $p \leq j \leq r$), the following holds:

1. $A[p:i]$ contains elements $\leq x$ (pivot)
2. $A[i+1:j-1]$ contains elements $> x$
3. $A[j:r-1]$ contains elements not yet processed
4. $A[r] = x$ (pivot)

Proof:

Initialization: Before first iteration ($j = p$), $i = p-1$. $A[p:p-1]$ is empty (trivially all $\leq x$). $A[p:p-1]$ is empty (trivially all $> x$). $A[p:r-1]$ is unprocessed. Invariant holds.

Maintenance: If $A[j] > x$, increment j . $A[i+1:j-1]$ now includes $A[j-1] > x$. Invariant holds. If $A[j] \leq x$, increment i and swap $A[i]$ with $A[j]$. Now $A[i] \leq x$ and $A[i+1:j-1]$ contains elements $> x$. Invariant holds.

Termination: When $j = r$, $A[p:i]$ contains elements $\leq x$, $A[i+1:r-1]$ contains elements $> x$. Swapping $A[i+1]$ with $A[r]$ places pivot in correct position. All elements left of pivot are $\leq$ it, all right are $>$ it. QED

QUICKSORT correctness: By induction on subarray size. Base case: 0 or 1 element is sorted. Inductive step: PARTITION correctly places pivot, then recursive calls sort left and right partitions. By IH, both are sorted, and pivot is between them. Thus entire array is sorted. QED

7. RANDOMIZED QUICKSORT

7.1 Algorithm Jist

- **Type:** Comparison-based, randomized divide-and-conquer sorting algorithm
- **How it works:** Same as Quicksort, but before partitioning, randomly selects an element and swaps it with the last element to use as pivot. This randomization prevents worst-case behavior on specific input patterns.
- **Best case to use:**
 - When input may be adversarial or nearly sorted
 - When worst-case $O(n^2)$ must be avoided with high probability
 - General-purpose sorting where average-case guarantee is preferred

7.2 Complexity Analysis

RANDOMIZED-PARTITION with Costs:

```
RANDOMIZED-PARTITION(A, p, r)
1  i = RANDOM(p, r)           -> c1
2  exchange A[r] with A[i]    -> c2
3  return PARTITION(A, p, r)   -> Theta(n)
```

RANDOMIZED-QUICKSORT:

```
RANDOMIZED-QUICKSORT(A, p, r)
1  if p < r                   -> c1
2      q = RANDOMIZED-PARTITION(A, p, r) -> Theta(n)
3      RANDOMIZED-QUICKSORT(A, p, q-1)   -> T(k)
4      RANDOMIZED-QUICKSORT(A, q+1, r)   -> T(n-k-1)
```

Expected Time Analysis:

- Expected running time $E[T(n)]$ satisfies: $E[T(n)] = (1/n) \sum [E[T(k)] + E[T(n-k-1)]] + \Theta(n)$ for $k = 0$ to $n-1$
- By symmetry: $E[T(n)] = (2/n) \sum E[T(k)]$ from $k=0$ to $n-1$ + $\Theta(n)$
- Solving via substitution: Assume $E[T(n)] \leq a \log(n)$
- After algebraic manipulation: $E[T(n)] = O(n \log n)$

Worst Case: Still $\Theta(n^2)$, but occurs with very low probability ($1/n!$ for specific adversarial inputs).

7.3 Properties

Stable: NO

- **Reason:** Same as Quicksort. The random swap and subsequent PARTITION exchanges can change relative order of equal elements.

In-place: YES

- **Reason:** Same as Quicksort. Only $O(1)$ extra space for swapping plus $O(\log n)$ expected stack space for recursion.

Adaptive: NO

- **Reason:** Randomization makes running time independent of input order in expectation. No performance improvement for partially sorted input.

7.4 Visual Example

Initial Array: $A = [3, 6, 2, 8, 1, 4, 7]$, $p = 1$, $r = 7$

First RANDOMIZED-PARTITION:

- $\text{RANDOM}(1, 7)$ returns, say, 3
- Swap $A[7]$ (7) with $A[3]$ (2) $\rightarrow [3, 6, 7, 8, 1, 4, 2]$
- Now call PARTITION with pivot = $A[7] = 2$
- $x = 2$, $i = 0$

j = 1: $A[1] = 3$ not $\leq 2 \rightarrow$ no change **j = 2:** $A[2] = 6$ not $\leq 2 \rightarrow$ no change **j = 3:** $A[3] = 7$ not $\leq 2 \rightarrow$ no change **j = 4:** $A[4] = 8$ not $\leq 2 \rightarrow$ no change **j = 5:** $A[5] = 1 \leq 2 \rightarrow i = 1$, swap $A[1]$ (3) with $A[5]$ (1) $\rightarrow [1, 6, 7, 8, 3, 4, 2]$ **j = 6:** $A[6] = 4$ not $\leq 2 \rightarrow$ no change

Swap $A[1]$ (1) with $A[7]$ (2) $\rightarrow [1, 2, 7, 8, 3, 4, 6]$

- Return $q = 2$

Recursive calls:

- $\text{RANDOMIZED-QUICKSORT}(A, 1, 1)$: [1] $\rightarrow$ done
- $\text{RANDOMIZED-QUICKSORT}(A, 3, 7)$: [7, 8, 3, 4, 6]
- $\text{RANDOM}(3, 7)$ returns, say, 5
- Swap $A[7]$ (6) with $A[5]$ (3) $\rightarrow [7, 8, 6, 4, 3]$
- PARTITION with pivot = 3
- $x = 3$, $i = 2$
- $j = 3$: 7 not $\leq 3 \rightarrow$ no change
- $j = 4$: 8 not $\leq 3 \rightarrow$ no change
- $j = 5$: 6 not $\leq 3 \rightarrow$ no change
- $j = 6$: 4 not $\leq 3 \rightarrow$ no change
- Swap $A[3]$ (7) with $A[7]$ (3) $\rightarrow [3, 8, 6, 4, 7]$

- Return $q = 3$
- RANDOMIZED-QUICKSORT($A, 3, 2$): empty
- RANDOMIZED-QUICKSORT($A, 4, 7$): $[8, 6, 4, 7]$
- Continue recursively...

Final sorted array: $[1, 2, 3, 4, 6, 7, 8]$

7.5 Loop Invariant and Correctness Proof

Same as Quicksort for PARTITION loop invariant.

Randomized Quicksort correctness:

- RANDOMIZED-PARTITION produces a random pivot, but once chosen, PARTITION behaves deterministically.
- The random choice does not affect the correctness of the partitioning.
- By the same inductive argument as Quicksort, RANDOMIZED-QUICKSORT correctly sorts any input.
- The randomization only affects the probability distribution of running times, not correctness. QED

8. COUNTING SORT

8.1 Algorithm Jist

- **Type:** Non-comparison-based, linear time sorting algorithm
- **How it works:** Counts the number of occurrences of each distinct element, computes cumulative counts to determine positions, then places elements in output array based on these positions.
- **Best case to use:**
 - When range of input $k = O(n)$
 - Integer sorting with small range
 - As subroutine in Radix Sort
 - When stability is required with linear time

8.2 Complexity Analysis

Algorithm with Costs:

```

COUNTING-SORT( $A, n, k$ )
1  let  $B[1:n]$  and  $C[0:k]$  be new arrays      -> c1
2  for  $i = 0$  to  $k$                              -> c2 ( $k + 1$  times)
3       $C[i] = 0$                                 -> c3 ( $k + 1$  times)
4  for  $j = 1$  to  $n$                              -> c4 ( $n + 1$  times)
5       $C[A[j]] = C[A[j]] + 1$                   -> c5 ( $n$  times)
7  for  $i = 1$  to  $k$                              -> c6 ( $k + 1$  times)
8       $C[i] = C[i] + C[i - 1]$                  -> c7 ( $k$  times)
11 for  $j = n$  downto 1                         -> c8 ( $n + 1$  times)
12      $B[C[A[j]]] = A[j]$                      -> c9 ( $n$  times)
13      $C[A[j]] = C[A[j]] - 1$                  -> c10 ( $n$  times)

```

Time Complexity:

- Line 2-3: $\Theta(k)$
- Line 4-5: $\Theta(n)$
- Line 7-8: $\Theta(k)$
- Line 11-13: $\Theta(n)$
- $T(n, k) = \Theta(n + k)$

When $k = O(n)$: $T(n) = \Theta(n)$

Space Complexity:

- Arrays B (size n) and C (size $k+1$)
- $S(n, k) = \Theta(n + k)$

8.3 Properties**Stable: YES**

- **Reason:** The algorithm processes A from right to left (line 11: $j = n$ down to 1) and places elements in B using cumulative counts. Elements with equal values appear in B in the same relative order as in A because later occurrences are placed first (due to decrementing $C[A[j]]$).

In-place: NO

- **Reason:** Requires two auxiliary arrays: B (output, size n) and C (counts, size $k+1$). Extra space is $\Theta(n + k)$.

Adaptive: NO

- **Reason:** Always performs the same operations regardless of input order. The counting, cumulative sum, and placement steps are executed identically for any input of size n with range k .

8.4 Visual Example

Initial Array: $A = [4, 2, 2, 8, 3, 3, 1]$, $n = 7$, $k = 8$

Step 1: Initialize C to 0

- $C = [0, 0, 0, 0, 0, 0, 0, 0, 0]$ (indices 0 to 8)

Step 2: Count occurrences

- $j = 1$: $A[1] = 4 \rightarrow C[4] = 1$
- $j = 2$: $A[2] = 2 \rightarrow C[2] = 1$
- $j = 3$: $A[3] = 2 \rightarrow C[2] = 2$
- $j = 4$: $A[4] = 8 \rightarrow C[8] = 1$
- $j = 5$: $A[5] = 3 \rightarrow C[3] = 1$
- $j = 6$: $A[6] = 3 \rightarrow C[3] = 2$

- $j = 7$: $A[7] = 1 \rightarrow C[1] = 1$

C after counting: $C = [0, 1, 2, 2, 1, 0, 0, 0, 1]$

Step 3: Cumulative counts

- $i = 1$: $C[1] = C[1] + C[0] = 1 + 0 = 1$
- $i = 2$: $C[2] = C[2] + C[1] = 2 + 1 = 3$
- $i = 3$: $C[3] = C[3] + C[2] = 2 + 3 = 5$
- $i = 4$: $C[4] = C[4] + C[3] = 1 + 5 = 6$
- $i = 5$: $C[5] = C[5] + C[4] = 0 + 6 = 6$
- $i = 6$: $C[6] = C[6] + C[5] = 0 + 6 = 6$
- $i = 7$: $C[7] = C[7] + C[6] = 0 + 6 = 6$
- $i = 8$: $C[8] = C[8] + C[7] = 1 + 6 = 7$

C after cumulative: $C = [0, 1, 3, 5, 6, 6, 6, 6, 7]$

(C[i] now contains number of elements $\leq i$)

Step 4: Place elements in B (right to left)

- $j = 7$: $A[7] = 1, C[1] = 1 \rightarrow B[1] = 1, C[1] = 0$
- $B = [1, *, *, *, *, *, *]$
- $j = 6$: $A[6] = 3, C[3] = 5 \rightarrow B[5] = 3, C[3] = 4$
- $B = [1, *, *, *, *, 3, *, _]$
- $j = 5$: $A[5] = 3, C[3] = 4 \rightarrow B[4] = 3, C[3] = 3$
- $B = [1, *, *, 3, 3, *, *]$
- $j = 4$: $A[4] = 8, C[8] = 7 \rightarrow B[7] = 8, C[8] = 6$
- $B = [1, *, *, 3, 3, _, 8]$
- $j = 3$: $A[3] = 2, C[2] = 3 \rightarrow B[3] = 2, C[2] = 2$
- $B = [1, *, 2, 3, 3, *, 8]$
- $j = 2$: $A[2] = 2, C[2] = 2 \rightarrow B[2] = 2, C[2] = 1$
- $B = [1, 2, 2, 3, 3, _, 8]$
- $j = 1$: $A[1] = 4, C[4] = 6 \rightarrow B[6] = 4, C[4] = 5$
- **$B = [1, 2, 2, 3, 3, 4, 8]$**

8.5 Loop Invariant and Correctness Proof

Loop Invariant for final for loop ($j = n$ downto 1): At the start of each iteration, B contains the correctly positioned elements from $A[j+1:n]$, and C contains the correct remaining counts for placing elements from $A[1:j]$.

Proof:

Initialization: Before first iteration ($j = n$), B is empty. C contains cumulative counts of all elements. Invariant holds vacuously.

Maintenance: At iteration j , $C[A[j]]$ contains the correct final position for $A[j]$ (number of elements $\leq A[j]$ that haven't been placed yet). Placing $A[j]$ at $B[C[A[j]]]$ and decrementing $C[A[j]]$ ensures the next occurrence of the same value goes to the position immediately before. This maintains stability and correct positioning.

Termination: When $j = 0$, all elements from $A[1:n]$ have been placed in B. By the invariant, all are in correct sorted positions. B is returned as the sorted array. QED

9. MINIMUM ELEMENT

9.1 Algorithm Jist

- **Type:** Selection algorithm, linear search
- **How it works:** Initializes min with first element, then scans through remaining elements updating min whenever a smaller element is found.
- **Best case to use:**
 - Finding minimum in an unsorted array
 - When only minimum is needed (not full sort)
 - Any scenario requiring a single linear scan

9.2 Complexity Analysis

Algorithm with Costs:

```
MINIMUM(A, n)
1  min = A[1]                -> c1
2  for i = 2 to n             -> c2 (n times)
3      if min > A[i]          -> c3 (n-1 times)
4          min = A[i]         -> c4 (t times)
5  return min                 -> c5
```

Where t = number of times a new minimum is found ($0 \leq t \leq n-1$).

Worst Case (Decreasing Order):

- New minimum found at every comparison
- $t = n-1$
- $T(n) = c1 + c2n + c3(n-1) + c4*(n-1) + c5$
- $T(n) = (c2 + c3 + c4)*n + (c1 - c3 - c4 + c5)$
- **$T(n) = \Theta(n)$**

Average Case:

- Expected number of updates: $H_{n-1} \sim \ln(n)$ (harmonic number)
- $T(n) = c1 + c2n + c3(n-1) + c4*H_{n-1} + c5$
- **$T(n) = \Theta(n)$**

Best Case (Increasing Order):

- No updates needed
- $t = 0$
- $T(n) = c_1 + c_2n + c_3(n-1) + c_5$
- $T(n) = \Theta(n)$

9.3 Properties**Stable: N/A**

- **Reason:** Not a sorting algorithm. Stability is not applicable to selection algorithms.

In-place: YES

- **Reason:** Uses only $O(1)$ extra space (variable min). No auxiliary arrays needed.

Adaptive: NO

- **Reason:** Always scans all $n-1$ elements regardless of input order. The number of comparisons is always $n-1$.

9.4 Visual Example

Initial Array: $A = [45, 12, 78, 23, 67, 8, 34]$, $n = 7$

Step 1: $\text{min} = A[1] = 45$

i = 2: $A[2] = 12$, $\text{min} = 45 > 12 \rightarrow \text{min} = 12$

i = 3: $A[3] = 78$, $\text{min} = 12$ not greater than 78 $\rightarrow$ min remains 12

i = 4: $A[4] = 23$, $\text{min} = 12$ not greater than 23 $\rightarrow$ min remains 12

i = 5: $A[5] = 67$, $\text{min} = 12$ not greater than 67 $\rightarrow$ min remains 12

i = 6: $A[6] = 8$, $\text{min} = 12 > 8 \rightarrow \text{min} = 8$

i = 7: $A[7] = 34$, $\text{min} = 8$ not greater than 34 $\rightarrow$ min remains 8

Return min = 8

9.5 Loop Invariant and Correctness Proof

Loop Invariant: At the start of each iteration i (where $2 \leq i \leq n+1$), min contains the minimum value among $A[1:i-1]$.

Proof:

Initialization: Before first iteration ($i = 2$), $\text{min} = A[1]$. The subarray $A[1:1]$ contains one element, which is trivially the minimum. Invariant holds.

Maintenance: Assume min is minimum of $A[1:i-1]$. At iteration i , we compare min with $A[i]$. If $A[i] < \text{min}$, then $A[i]$ is the new minimum of $A[1:i]$. If not, min remains the minimum of $A[1:i]$. In either case, min is minimum of $A[1:i]$.

Termination: When $i = n+1$, min is the minimum of $A[1:n]$, which is the entire array. Thus min is the global minimum. QED

10. SIMULTANEOUS MIN-MAX

10.1 Algorithm Jist

- **Type:** Selection algorithm, optimized pairwise comparison
- **How it works:** Processes elements in pairs, comparing each pair first, then comparing the smaller with current min and larger with current max. Reduces total comparisons from $2n$ to $3n/2$.
- **Best case to use:**
 - When both minimum and maximum are needed simultaneously
 - Tournament structures
 - Range queries on unsorted data
 - Any scenario where min and max are both required

10.2 Complexity Analysis

Algorithm with Costs:

```
SIMULTANEOUS-MIN-MAX(A, n)
1   if n is odd                      -> c1
2       min = A[1]                   -> c2
3       max = A[1]                   -> c3
4       i = 2                        -> c4
5   else
6       if A[1] < A[2]                -> c5
7           min = A[1]                -> c6
8           max = A[2]                -> c7
9       else
10          min = A[2]                 -> c8
11          max = A[1]                 -> c9
12          i = 3                     -> c10
13  while i <= n - 1                  -> c11 (n/2 times)
14      if A[i] < A[i+1]               -> c12 (n/2 - 1 times)
15          if A[i] < min               -> c13
16              min = A[i]              -> c14
17          if A[i+1] > max             -> c15
18              max = A[i+1]            -> c16
19      else
20          if A[i+1] < min              -> c17
21              min = A[i+1]            -> c18
22          if A[i] > max                -> c19
23              max = A[i]              -> c20
24      i = i + 2                      -> c21
25  return (min, max)                 -> c22
```

Comparison Count Analysis: If n is odd:

- Initial: 0 comparisons
- Loop: 3 comparisons per pair ($A[i]$ vs $A[i+1]$, winner vs max, loser vs min)
- Number of pairs: $(n-1)/2$

- Total comparisons: $3(n-1)/2 = \text{ceil}(3n/2) - 2$

If n is even:

- Initial: 1 comparison ($A[1]$ vs $A[2]$)
- Loop: 3 comparisons per pair
- Number of pairs: $(n-2)/2$
- Total comparisons: $1 + 3(n-2)/2 = 3n/2 - 2$

In both cases:

- $T(n) = \Theta(n)$ comparisons (approximately $3n/2$)
- $T(n) = \Theta(n)$ time overall

Note: Naive approach uses $2n-2$ comparisons. This saves $n/2-2$ comparisons.

10.3 Properties

Stable: N/A

- **Reason:** Not a sorting algorithm. Stability is not applicable.

In-place: YES

- **Reason:** Uses only $O(1)$ extra space (variables: min, max, i). No auxiliary arrays needed.

Adaptive: NO

- **Reason:** Always processes elements in pairs regardless of input order. The number of comparisons is fixed at approximately $3n/2$.

10.4 Visual Example

Initial Array: $A = [3, 8, 2, 9, 5, 1, 7, 4]$, $n = 8$ (even)

Initialization (n is even):

- $A[1] = 3 < A[2] = 8 \rightarrow \text{min} = 3, \text{max} = 8$
- $i = 3$

Pair 1 (i = 3): $A[3] = 2, A[4] = 9$

- $2 < 9 \rightarrow \text{yes}$
- $2 < \text{min}(3)$? Yes $\rightarrow \text{min} = 2$
- $9 > \text{max}(8)$? Yes $\rightarrow \text{max} = 9$
- $i = 5$

Pair 2 (i = 5): $A[5] = 5, A[6] = 1$

- $5 < 1$? No $\rightarrow$ else branch
- $1 < \text{min}(2)$? Yes $\rightarrow \text{min} = 1$
- $5 > \text{max}(9)$? No $\rightarrow$ max remains 9

- $i = 7$

Pair 3 ($i = 7$): $A[7] = 7$, $A[8] = 4$

- $7 < 4$? No $\rightarrow$ else branch
- $4 < \min(1)$? No $\rightarrow$ min remains 1
- $7 > \max(9)$? No $\rightarrow$ max remains 9
- $i = 9$

Return $(\min, \max) = (1, 9)$

10.5 Loop Invariant and Correctness Proof

Loop Invariant: At the start of each while loop iteration (where $i \leq n-1$), min contains the minimum of $A[1:i-1]$ and max contains the maximum of $A[1:i-1]$.

Proof:

Initialization: Before first while iteration, if n is even, we compare $A[1]$ and $A[2]$, setting min and max correctly for $A[1:2]$. If n is odd, $\min = \max = A[1]$ for $A[1:1]$. In both cases, invariant holds.

Maintenance: At each iteration, we compare $A[i]$ and $A[i+1]$. The smaller is compared with min and the larger with max. After processing the pair, min is the minimum of $A[1:i+1]$ and max is the maximum of $A[1:i+1]$. Incrementing i by 2 preserves invariant.

Termination: When $i > n-1$, all elements have been processed. If n is even, all n elements are in pairs. If n is odd, the last element $A[n]$ was already included in the initialization. Thus min and max are correct for the entire array. QED

11. STRASSEN'S MATRIX MULTIPLICATION

11.1 Algorithm Jist

- **Type:** Divide-and-conquer matrix multiplication algorithm
- **How it works:** Recursively divides $n \times n$ matrices into $n/2 \times n/2$ submatrices, then uses 7 clever multiplications (instead of 8 in standard divide-and-conquer) to compute the product, reducing complexity from $O(n^3)$ to $O(n^{2.81})$.
- **Best case to use:**
 - Large dense matrix multiplication
 - When matrix dimensions are powers of 2
 - Theoretical applications where lower exponent matters
 - When standard $O(n^3)$ is too slow for large matrices

11.2 Complexity Analysis

Supporting Functions: **MATRIX-ADD(A, B):**

```

1  n = A.rows
2  let C be a new n x n matrix
3  for i = 1 to n                -> c1 (n+1 times)
4      for j = 1 to n            -> c2 (n*(n+1) times)
5          C[i, j] = A[i, j] + B[i, j]    -> c3 (n^2 times)
```

```
6   return C
```

Time: $\Theta(n^2)$

MATRIX-SUB(A, B):

```
1  n = A.rows
2  let C be a new n x n matrix
3  for i = 1 to n                -> c1 (n+1 times)
4      for j = 1 to n            -> c2 (n*(n+1) times)
5          C[i, j] = A[i, j] - B[i, j]    -> c3 (n^2 times)
6  return C
```

Time: $\Theta(n^2)$

STRASSEN Algorithm with Costs:

```
STRASSEN(A, B)
1  n = A.rows                -> c1
2  let C be a new n x n matrix    -> c2
3  if n == 1                    -> c3
4      C[1, 1] = A[1, 1] * B[1, 1]    -> c4
5      return C                    -> c5
6  partition A into n/2 x n/2 submatrices A11, A12, A21, A22
7  partition B into n/2 x n/2 submatrices B11, B12, B21, B22
8  S1 = MATRIX-SUB(B12, B22)        -> Theta(n^2)
9  S2 = MATRIX-ADD(A11, A12)         -> Theta(n^2)
10 S3 = MATRIX-ADD(A21, A22)         -> Theta(n^2)
11 S4 = MATRIX-SUB(B21, B11)         -> Theta(n^2)
12 S5 = MATRIX-ADD(A11, A22)         -> Theta(n^2)
13 S6 = MATRIX-ADD(B11, B22)         -> Theta(n^2)
14 S7 = MATRIX-SUB(A12, A22)         -> Theta(n^2)
15 S8 = MATRIX-ADD(B21, B22)         -> Theta(n^2)
16 S9 = MATRIX-SUB(A11, A21)         -> Theta(n^2)
17 S10 = MATRIX-ADD(B11, B12)        -> Theta(n^2)
18 P1 = STRASSEN(A11, S1)            -> T(n/2)
19 P2 = STRASSEN(S2, B22)            -> T(n/2)
20 P3 = STRASSEN(S3, B11)            -> T(n/2)
21 P4 = STRASSEN(A22, S4)            -> T(n/2)
22 P5 = STRASSEN(S5, S6)             -> T(n/2)
23 P6 = STRASSEN(S7, S8)             -> T(n/2)
24 P7 = STRASSEN(S9, S10)            -> T(n/2)
25 C11 = MATRIX-ADD(MATRIX-SUB(MATRIX-ADD(P5, P4), P2), P6) -> Theta(n^2)
26 C12 = MATRIX-ADD(P1, P2)          -> Theta(n^2)
27 C21 = MATRIX-ADD(P3, P4)          -> Theta(n^2)
28 C22 = MATRIX-SUB(MATRIX-SUB(MATRIX-ADD(P5, P1), P3), P7) -> Theta(n^2)
29 combine C11, C12, C21, C22 into matrix C    -> Theta(n^2)
30 return C                                -> c6
```

Recurrence Relation: $T(n) = \Theta(1)$ if $n = 1$ $T(n) = 7T(n/2) + \Theta(n^2)$ if $n > 1$

Explanation:

- 7 recursive calls on $n/2 \times n/2$ matrices
- $\Theta(n^2)$ work for additions/subtractions (10 additions/subtractions of $n/2 \times n/2$ matrices, plus combining results)

Solving by Master Theorem: $a = 7$, $b = 2$, $f(n) = \Theta(n^2)$ $n^{\log_b a} = n^{\log_2 7} \sim n^{2.81}$
 Since $f(n) = \Theta(n^2) = O(n^{\log_2 7 - \epsilon})$ for $\epsilon \sim 0.81$, Case 1 applies.

$$T(n) = \Theta(n^{\log_2 7}) = \Theta(n^{2.807})$$

Note: $\log_2(7) \sim 2.807$, which is less than 3 (standard matrix multiplication).

11.3 Properties

Stable: N/A

- **Reason:** Not a sorting algorithm. Stability is not applicable to matrix operations.

In-place: NO

- **Reason:** Requires multiple auxiliary matrices (S1-S10, P1-P7, and submatrices). Extra space is $\Theta(n^2)$ for the recursive stack and temporary matrices.

Adaptive: NO

- **Reason:** Always performs the same 7 recursive multiplications and $\Theta(n^2)$ additions regardless of input matrix values.

11.4 Visual Example

Matrices: $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$, $n = 2$

Base Case ($n = 2$, which is power of 2):

Partitions:

- $A_{11} = [1]$, $A_{12} = [2]$, $A_{21} = [3]$, $A_{22} = [4]$
- $B_{11} = [5]$, $B_{12} = [6]$, $B_{21} = [7]$, $B_{22} = [8]$

Compute S matrices:

- $S_1 = B_{12} - B_{22} = [6] - [8] = [-2]$
- $S_2 = A_{11} + A_{12} = [1] + [2] = [3]$
- $S_3 = A_{21} + A_{22} = [3] + [4] = [7]$
- $S_4 = B_{21} - B_{11} = [7] - [5] = [2]$
- $S_5 = A_{11} + A_{22} = [1] + [4] = [5]$
- $S_6 = B_{11} + B_{22} = [5] + [8] = [13]$
- $S_7 = A_{12} - A_{22} = [2] - [4] = [-2]$
- $S_8 = B_{21} + B_{22} = [7] + [8] = [15]$
- $S_9 = A_{11} - A_{21} = [1] - [3] = [-2]$
- $S_{10} = B_{11} + B_{12} = [5] + [6] = [11]$

Recursive multiplications (base case n=1, so direct multiplication):

- $P1 = A11 * S1 = [1] * [-2] = [-2]$
- $P2 = S2 * B22 = [3] * [8] = [24]$
- $P3 = S3 * B11 = [7] * [5] = [35]$
- $P4 = A22 * S4 = [4] * [2] = [8]$
- $P5 = S5 * S6 = [5] * [13] = [65]$
- $P6 = S7 * S8 = [-2] * [15] = [-30]$
- $P7 = S9 * S10 = [-2] * [11] = [-22]$

Compute C submatrices:

- $C11 = P5 + P4 - P2 + P6 = [65] + [8] - [24] + [-30] = [19]$
- $C12 = P1 + P2 = [-2] + [24] = [22]$
- $C21 = P3 + P4 = [35] + [8] = [43]$
- $C22 = P5 + P1 - P3 - P7 = [65] + [-2] - [35] - [-22] = [50]$

Result: $C = [[19, 22], [43, 50]]$

Verification: Standard multiplication:

- $C[1,1] = 15 + 27 = 5 + 14 = 19$
- $C[1,2] = 16 + 28 = 6 + 16 = 22$
- $C[2,1] = 35 + 47 = 15 + 28 = 43$
- $C[2,2] = 36 + 48 = 18 + 32 = 50$

Matches!

11.5 Loop Invariant and Correctness Proof

Correctness Proof:

We prove that STRASSEN correctly computes $C = A * B$ by showing that the formulas for C11, C12, C21, C22 are algebraically equivalent to standard matrix multiplication.

Standard multiplication:

- $C11 = A11B11 + A12B21$
- $C12 = A11B12 + A12B22$
- $C21 = A21B11 + A22B21$
- $C22 = A21B12 + A22B22$

Strassen's formulas:

- $C11 = P5 + P4 - P2 + P6 = (A11+A22)(B11+B22) + A22(B21-B11) - (A11+A12)B22 + (A12-A22)(B21+B22) = A11B11 + A11B22 + A22B11 + A22B22 + A22B21 - A22B11 - A11B22 - A12B22 + A12B21 + A12B22 - A22B21 - A22B22 = A11B11 + A12B21$ [all other terms cancel]

- $C12 = P1 + P2 = A11(B12-B22) + (A11+A12)B22 = A11B12 - A11B22 + A11B22 + A12B22 = A11B12 + A12B22$
- $C21 = P3 + P4 = (A21+A22)B11 + A22(B21-B11) = A21B11 + A22B11 + A22B21 - A22B11 = A21B11 + A22B21$
- $C22 = P5 + P1 - P3 - P7 = (A11+A22)(B11+B22) + A11(B12-B22) - (A21+A22)B11 - (A11-A21)(B11+B12) = A11B11 + A11B22 + A22B11 + A22B22 + A11B12 - A11B22 - A21B11 - A22B11 - A11B11 - A11B12 + A21B11 + A21B12 = A21B12 + A22B22$ [all other terms cancel]

All four submatrices match the standard definition. By induction on matrix size (base case $n=1$ is direct multiplication, inductive step assumes correctness for $n/2$), STRASSEN is correct. QED

SUMMARY TABLE

Algorithm	Worst Case	Average Case	Best Case	Stable	In-Place	Adaptive
Insertion Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$	Yes	Yes	Yes
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	No	Yes	No
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	Yes	No	No
Bubble Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	Yes	Yes	No*
Heapsort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	No	Yes	No
Quicksort	$\Theta(n^2)$	$\Theta(n \log n)$	$\Theta(n \log n)$	No	Yes	No
Randomized Quicksort	$\Theta(n^2)^*$	$\Theta(n \log n)$	$\Theta(n \log n)$	No	Yes	No
Counting Sort	$\Theta(n+k)$	$\Theta(n+k)$	$\Theta(n+k)$	Yes	No	No
Minimum	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	N/A	Yes	No
Simultaneous Min-Max	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	N/A	Yes	No
Strassen	$\Theta(n^{2.807})$	$\Theta(n^{2.807})$	$\Theta(n^{2.807})$	N/A	No	No

Bubble Sort: Adaptive with optimization flag (best case $\Theta(n)$) Randomized Quicksort: Worst case occurs with very low probability

END OF UNIT 1 NOTES